

Encoding, normalización y parsers

Objetivos

Distinguir representación de significado, ordenar transformaciones y explicar por qué dos componentes pueden interpretar los mismos bytes de modo diferente.

Encoding no es cifrado

Base64, percent-encoding, Unicode y escapes JSON representan datos; no prueban confidencialidad. Decodificar recupera otra representación sin secreto. El significado aparece cuando un parser consume el resultado.

Ejemplo conceptual:

```
entrada HTTP:      %2e%2e%2fconfig
decodificación URL: ../config
normalización ruta: /srv/config
operación final:   open('/srv/config')
```

Bloquear la primera cadena y ejecutar la tercera es validación en una fase incorrecta.

Orden de procesamiento

Documenta siempre:

```
bytes -> decodificación de transporte -> parser de formato -> coerción de tipo
      -> normalización/canonicalización -> validación -> parser final -> sink
```

No presupongas el orden. Una aplicación puede validar antes de decodificar, decodificar dos veces o delegar la normalización al sistema de ficheros.

Parser, intérprete y sink

- Un parser convierte una representación en estructura: JSON a objetos, URL a componentes, SQL a árbol sintáctico.
- Un intérprete asigna comportamiento a esa estructura: shell, motor SQL o plantilla.
- Un sink es una operación sensible: ejecutar, consultar, abrir fichero, realizar petición o decidir acceso.

La vulnerabilidad aparece cuando datos controlados alteran estructura o selección de recurso en el sink sin una frontera segura.

Validación textual frente a semántica

"127.0.0.1" no aparece es una comparación textual. la dirección resuelta no pertenece a loopback, privada ni metadatos es una condición semántica. La segunda debe aplicarse al destino final y repetirse tras redirecciones o cambios de resolución cuando corresponda.

Cómo diagnosticar

1. Parte de una entrada válida.
2. Cambia una sola representación.
3. Observa errores de capa: URL, JSON, plantilla, SQL, shell o filesystem.
4. Compara lo enviado con lo registrado y lo ejecutado cuando el laboratorio ofrece logs.
5. Determina cuántas decodificaciones ocurren.
6. Confirma el significado final con una evidencia inocua.

Error frecuente

Una variante codificada que funciona no demuestra por sí sola que "el filtro se saltó". Puede haber cambiado el transporte, el router o la selección de recurso. La conclusión debe señalar qué componente comparó qué forma.

Laboratorio: localizar la decodificación

Un endpoint recibe `file=notes%2Ftoday.txt` y abre `notes/today.txt`. Con `file=..%252Fsecret.txt` responde `blocked token: ..%2F`; con `file=%252e%252e%252fsecret.txt` intenta abrir `../secret.txt`.

Observación: dos entradas doblemente codificadas alcanzan fases distintas. Qué sé realmente: hay al menos dos representaciones y el mensaje de bloqueo muestra una de ellas. Hipótesis: H1, el filtro se ejecuta tras una primera decodificación y antes de una segunda; H2, dos componentes decodifican rutas distintas. Experimento: enviar marcadores donde solo uno de `%25`, `%2e` y `%2f` cambie, y comparar el valor registrado justo antes del `open()` si el laboratorio lo expone. Predicción: H1 mostrará una transformación consistente en todas las variantes; H2 producirá rutas o errores de componentes diferentes. Resultado del escenario: el log muestra `raw -> urldecode -> filtro -> router urldecode -> open`. Conclusión: no "funcionó el encoding"; la propiedad se validó antes de obtener la forma consumida.

Ejercicio de hipótesis rivales

`%2Fadmin` devuelve 404, mientras `/admin` devuelve 403. Diseña una prueba para separar: (a) el proxy no decodifica `%2F`; (b) el router lo trata como parte de un segmento; (c) existe una regla de autorización solo para la ruta canónica. No concluyas bypass hasta identificar qué ruta vio cada capa.

Referencias

- [RFC 3986. percent-encoding y normalización](#)
- [Fuente interna: filtros incompletos](#)

Anterior: [Redes](#). Siguiente: [Linux y shell](#).